plan

Master Plan: LLMsVerifier Integration into HelixCode

Objective

Create an in-depth, step-by-step integration plan for making LLMsVerifier the single source of truth for model provisioning in HelixTrack (HelixCode), with HelixAgent as the reference implementation.

Stage 1 — Deep Repository Analysis (Parallel Research Swarm)

Skill: `deep-research-swarm` (Route B — Focused Search) **Agents**: 1. **Repo_Analyzer_HelixCode** — Analyze HelixCode repository structure, configuration system, model management, CLI architecture, UX patterns, supported platforms, existing test framework, Challenges system, Constitution/CLAUDE.MD/AGENTS.MD presence. 2. **Repo_Analyzer_LLMV** — Analyze LLMsVerifier repository structure, verification engine, scoring algorithm, provider support, rate limiting logic, token tracking, pricing model, configuration requirements, API integration patterns. 3. **Repo_Analyzer_HelixAgent** — Analyze HelixAgent as reference implementation: how it integrated LLMsVerifier, configuration options, model display UX, enable/disable mechanisms, provider management, API key provisioning, real-time updates handling. 4. **Cross_Reference_Analyst** — Compare findings across repos, identify integration touchpoints, gaps between HelixAgent and HelixCode architectures, configuration schema differences. 5. **Documentation_Archaeologist** — Extract all documentation, README, guides, manuals, configuration references, CLAUDE.MD, AGENTS.MD, Constitution files from all repos.

Deliverable: Complete research brief with exact file paths, line references, configuration schemas, API contracts, and integration points.

Stage 2 — Architecture & Integration Planning

Skill: Orchestrator-designed (no skill needed — architecture synthesis) **Agents**: 1. **Integration_Architect** — Design the full integration architecture: configuration schema changes, module boundaries, API contracts between HelixCode and LLMsVerifier, data flow for real-time updates, model state management, caching strategy. 2. **UX_Designer** — Design the enterprise-grade model display UX for CLI (all platforms): model listing, validation badges, cooldown indicators, provider attribution, pricing display, rate limit status, filtering/sorting, help text. 3. **Provider_Integration_Specialist** — Map all providers and models, plan full incorporation of MCPs, LSPs, ACPs, Embeddings, RAGs, Skills, Plugins from all supported providers.

Deliverable: Architecture document, UX specification, provider integration matrix.

Stage 3 — Phased Implementation Planning

Skill: Orchestrator-designed **Agents**: 1. **Phase_Planner_Core** — Plan Phase 1: Foundation (config, dependency injection, basic integration) 2. **Phase_Planner_ModelMgmt** — Plan Phase 2: Model Management (LLMsVerifier as source of truth, real-time updates, state synchronization) 3. **Phase_Planner_UX** — Plan Phase 3: UX Implementation (model display, CLI integration, platform compatibility) 4. **Phase_Planner_AdvFeatures** — Plan Phase 4: Advanced Features (MCPs, LSPs, ACPs, Embeddings, RAGs, Skills, Plugins) 5. **Phase_Planner_Testing** — Plan Phase 5: Testing Strategy (unit, integration, e2e, Challenges, anti-bluff verification, 100% coverage target) 6. **Phase_Planner_Docs** — Plan Phase 6: Documentation & Constitution updates (all config options, guides, CLAUDE.MD, AGENTS.MD, Constitution files)

Deliverable: Six-phase implementation plan with fine-grained tasks, exact file references, line numbers, code snippets, and dependency chains.

Stage 4 — Quality Assurance & Constitution Integration

Agents: 1. **Constitution_Integrator** — Draft Constitution amendments, CLAUDE.MD and AGENTS.MD updates for anti-bluff testing guarantee 2. **Test_Strategist** — Design comprehensive testing matrix ensuring every feature is testable and tested

Deliverable: Constitution updates, testing manifesto, challenge specifications.

Stage 5 — Final Assembly

Agents: 1. **Plan_Assembler** — Combine all outputs into single comprehensive document 2. **Review_Verifier** — Verify completeness: no skipped sections, no assumptions without evidence, every claim backed by repo analysis

Deliverable: Final integration plan document (.md then .docx)

Output Format

- Markdown document with table of contents
- Converted to .docx for delivery
- All file paths, line numbers, exact code references
- Every phase with sub-tasks and acceptance criteria
- Testing strategy with 100% coverage blueprint
- Constitution/CLAUDE.MD/AGENTS.MD amendments